

LAB 4 - Create DAX calculations in semantic models

Lab story

In this lab, you'll create calculated tables, calculated columns, and simple measures by using Data Analysis Expressions (DAX).

In this lab, you learn how to:

- Create calculated tables.
- Create calculated columns.
- Create measures.

This lab should take approximately 45 minutes.

Get started

Extract the file to the `C:\Users\skill\OneDrive\Desktop\04-dax-calculations` folder.

Open the **04-Starter-Sales Analysis.pbix** file.

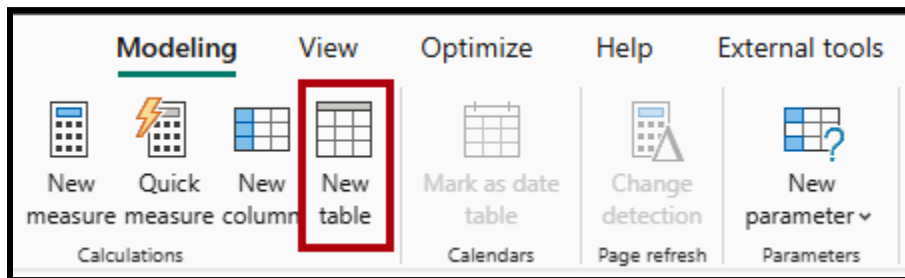
Create the Salesperson calculated table

In this task, you'll create the **Salesperson calculated table** (that will have a direct relationship to the Sales table).

A calculated table is created by first entering the **table name**, followed by the equals symbol (=), followed by a **DAX** formula that returns a table. The table name can't already exist in the data model.

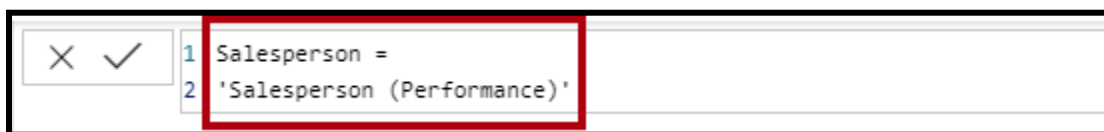
You enter a valid DAX formula in the *formula bar*. The formula bar includes features like auto-complete, Intellisense and color-coding, which allow you to quickly and accurately enter the formula.

1. In Power BI Desktop, in **Report view**, on the Modeling ribbon, from inside the Calculations group, select **New Table**.



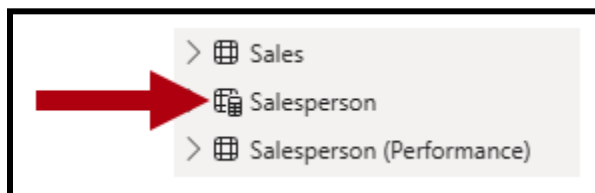
2. In the formula bar (which opens directly beneath the ribbon when you create or edit calculations), type **Salesperson =**, press **Shift+Enter**, type **'Salesperson (Performance)'**, and then press Enter.

*Note: For your convenience, all **DAX definitions** in this lab can be copied from the snippets file, located in the 04-dax-calculations\Snippets.txt file.*



This table definition creates a copy of the Salesperson (Performance) table. It copies the data only, however model properties like visibility, formatting, and others aren't copied.

3. In the Data pane, notice that the **icon** for the new table has an **additional calculator** in front of it (denoting a calculated table).



Note: Calculated tables are defined by using a DAX formula that returns a table. It's important to understand that calculated tables increase the size of the data model because they materialize and store values. Also, they're recomputed whenever formula dependencies are refreshed, as will be the case for this data model when new (future) date values are loaded into tables. Unlike Power Query-sourced tables, calculated tables can't be used to load data from external data sources. They can only transform data based on what has already been loaded into the data model.

4. Switch to **Model view**, and notice that the **Salesperson table** is available.

5. Create a relationship from the **Salesperson | EmployeeKey** column to the **Sales | EmployeeKey** column.

The labs use a shorthand notation to reference a field. It will look like this: Salesperson | EmployeeKey. In this example, Salesperson is the table name and EmployeeKey is the column name.

6. Right-click the **inactive relationship** (dotted line) between the **Salesperson (Performance) and Sales tables**, and then select **Delete**. When prompted to confirm the deletion, select Yes.

7. In the **Salesperson table**, multi-select the following columns, and then hide them (**set the Is Hidden property to Yes**):

- EmployeeID
- EmployeeKey
- UPN

8. Select the **Salesperson table**.

In the **Properties** pane, in the Description box, enter: **Salesperson related to sales.**

You may recall that descriptions appear as tooltips in the Data pane whenever the user hovers their cursor over a table or field.

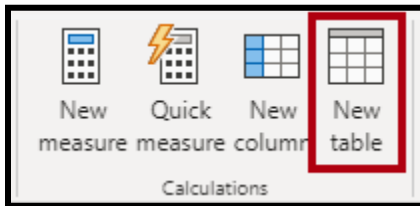
9. For the **Salesperson (Performance) table**, set the description to: **Salesperson related to region(s)**

The data model now provides two alternatives when analyzing salespeople. The Salesperson table allows analyzing sales made by a salesperson, while the Salesperson (Performance) table allows analyzing sales made in the sales region(s) assigned to the salesperson.

Create the Date table

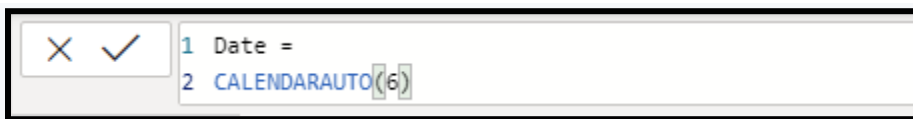
In this task, you'll create the Date table.

1. Switch to **Table view**. On the **Home** ribbon tab, from inside the Calculations group, select **New Table**.



2. In the formula bar, enter the following DAX:

Date = CALENDARAUTO(6)



The *CALENDARAUTO* function returns a single-column table comprising date values. The "auto" behavior scans all data model date columns to determine the earliest and latest date values stored in the data model. It then creates one row for each date within this range, extending the range in either direction to ensure full years of data is stored.

This function can take a single optional argument that is the last month number of a year. When omitted, the value is 12, meaning that December is the last month of the year. In this case, 6 is entered, meaning that June is the last month of the year.

3. Notice the column of date values which might be formatted using US regional settings (that is, mm/dd/yyyy).

Date
07/01/2017 00:00:00
07/02/2017 00:00:00
07/03/2017 00:00:00
07/04/2017 00:00:00
07/05/2017 00:00:00
07/06/2017 00:00:00

4. At the bottom-left corner, in the status bar, notice the table statistics, confirming that 1826 rows of data have been generated, which represents five full years' data.

TABLE: Date (1,826 rows)

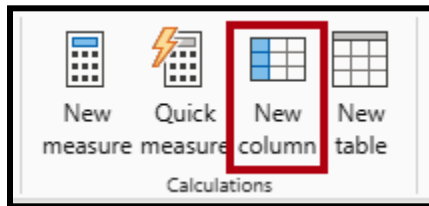
Create calculated columns

In this task, you'll add more columns to enable **filtering and grouping** by different time periods. You'll also create a calculated column to control the sort order of other columns.

Note: For your convenience, all DAX definitions in this lab can be copied from the Snippets.txt file.

- On the **Table Tools** contextual ribbon, from inside the Calculations group, select **New Column**.

A calculated column is created by first entering the column name, followed by the equals symbol (=), followed by a DAX formula that returns a single-value result. The column name can't already exist in the table.



- In the formula bar, type the following (or copy from the snippets file), and then press Enter:

The formula uses the date's year value but adds one to the year value when the month is after June. That's how fiscal years at Adventure Works are calculated.

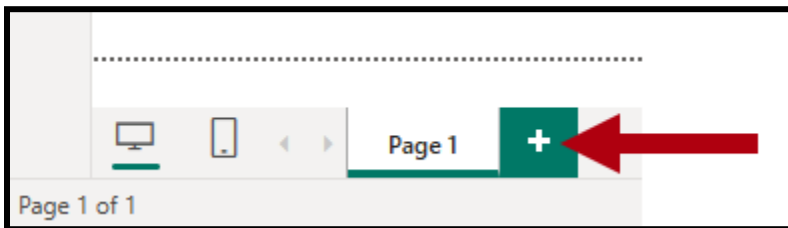
Year = "FY" & YEAR('Date'[Date]) + IF(MONTH('Date'[Date]) > 6, 1)

- Use the snippets file definitions to create the following two calculated columns for the Date table:
 - **Quarter** = "FY" & YEAR('Date'[Date]) + IF(MONTH('Date'[Date]) > 6, 1, 0) & " Q" & ROUNDUP(MOD(MONTH('Date'[Date]) - 7, 12) / 3 + 1, 0)
 - **Month** = YEAR('Date'[Date]) & " " & FORMAT('Date'[Date], "MMM")

- Verify the new columns have been added.

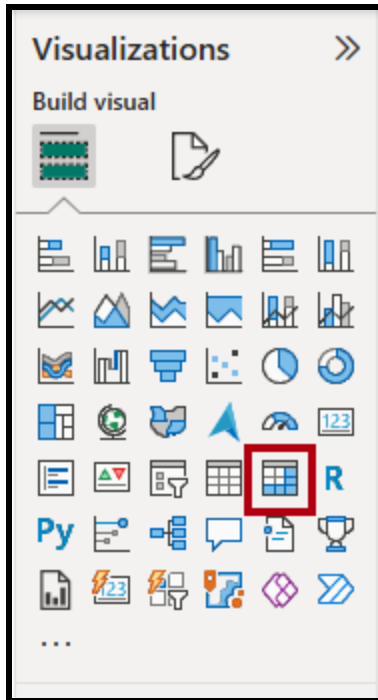
Date	Year	Quarter	Month
01/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
02/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
03/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
04/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
05/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
06/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul
07/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul

- To validate the calculations, switch to **Report view**.
- To create a new report page, select the plus icon next to Page 1.

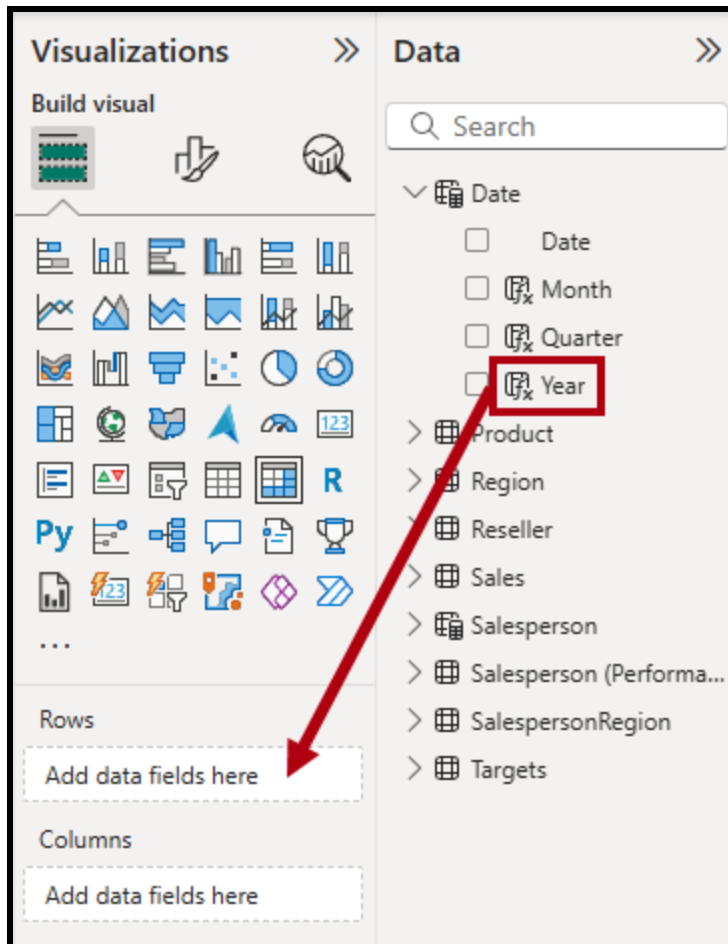


- To add a matrix visual to the new report page, in the **Visualizations** pane, select the **matrix visual type**.

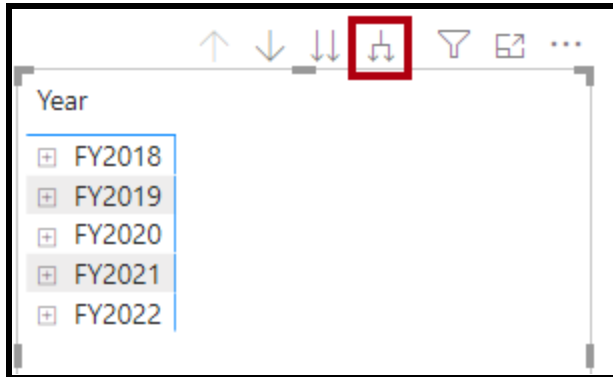
Tip: You can hover the cursor over each icon to reveal a tooltip describing the visual type.



- In the **Data pane**, from inside the **Date table**, drag the **Year field** into the Rows well.



- Drag the **Month** field into the Rows well, directly beneath the Year field.
- At the top-right of the matrix visual (or bottom, depending on the location of the visual), select the **forked-double arrow** icon (which will expand all years down one level).



- Notice that the years expand to months, and that the months are sorted alphabetically rather than chronologically.



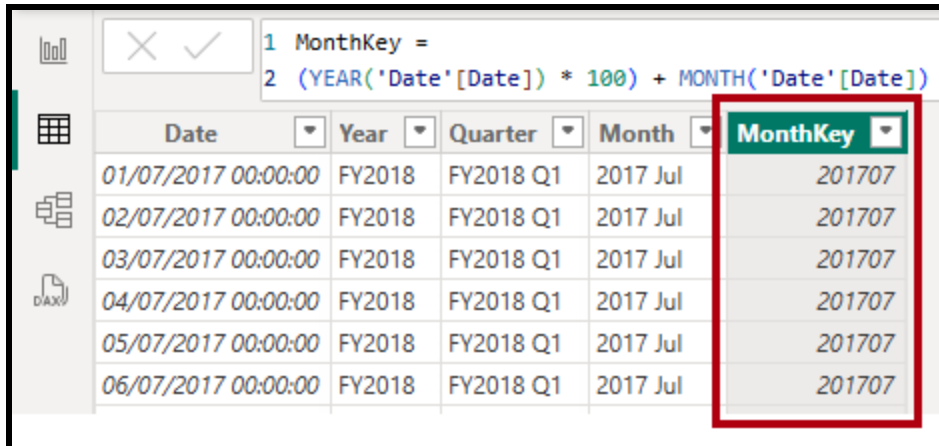
By default, text values sort alphabetically, numbers sort from smallest to largest, and dates sort from earliest to latest.

- To customize the Month field sort order, switch to Table view.
- Add the MonthKey column to the Date table.

MonthKey = (YEAR('Date'[Date]) * 100) + MONTH('Date'[Date]))

This formula computes a numeric value for each year/month combination.

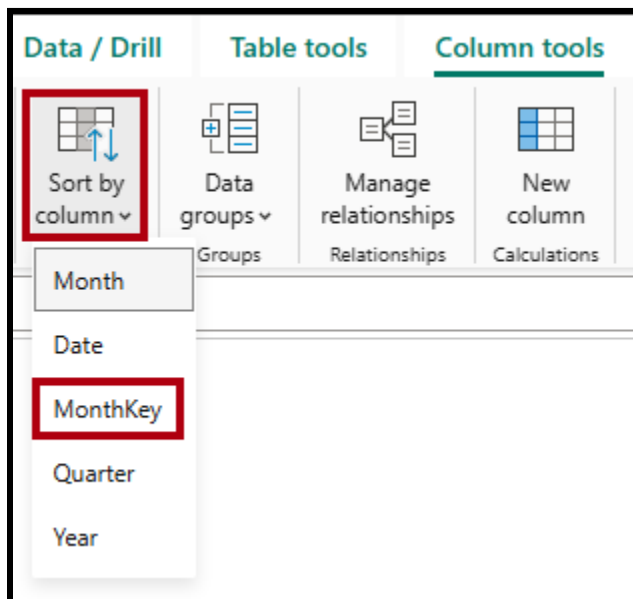
- In Table view, verify that the new column contains numeric values (for example, 201707 for July 2017, and so on).



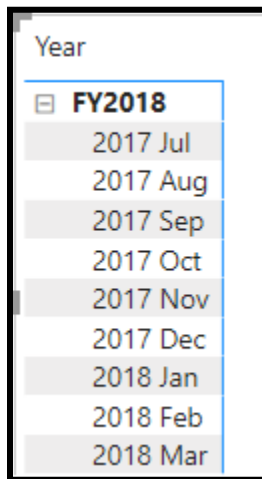
1 MonthKey =
2 (YEAR('Date'[Date]) * 100) + MONTH('Date'[Date]))

Date	Year	Quarter	Month	MonthKey
01/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
02/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
03/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
04/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
05/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707
06/07/2017 00:00:00	FY2018	FY2018 Q1	2017 Jul	201707

- Switch back to **Report view**.
- In the **Data pane**, select the **Month** field.
- On the **Column Tools** contextual ribbon, from inside the Sort group, select **Sort by Column**, and then select **MonthKey**.



- In the matrix visual, notice that the months are now chronologically sorted.

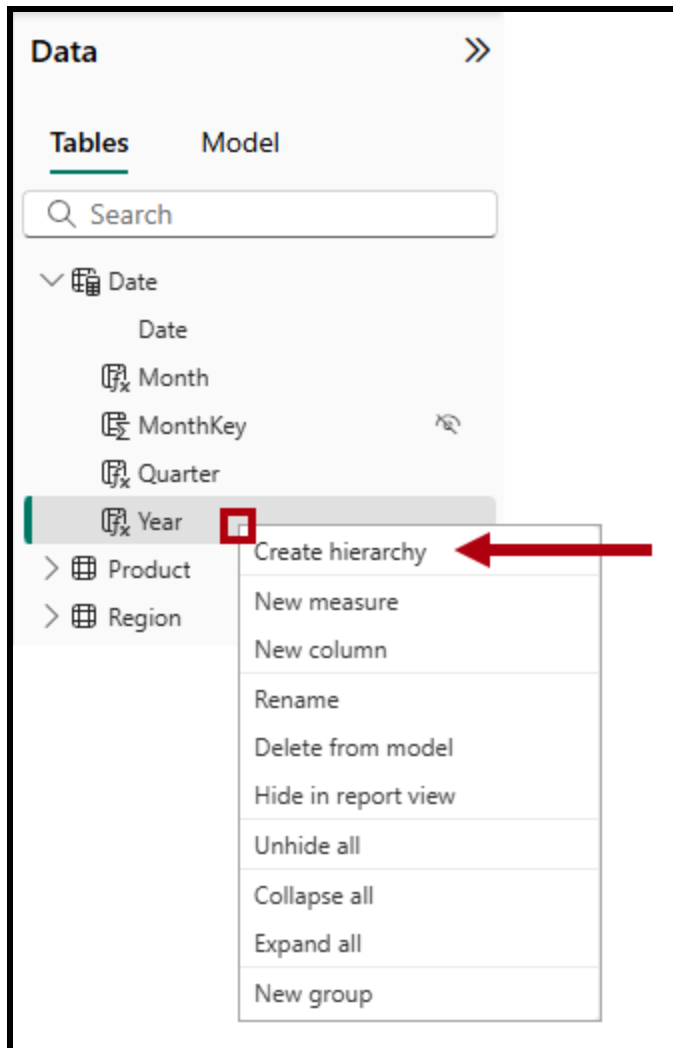


Year
FY2018
2017 Jul
2017 Aug
2017 Sep
2017 Oct
2017 Nov
2017 Dec
2018 Jan
2018 Feb
2018 Mar

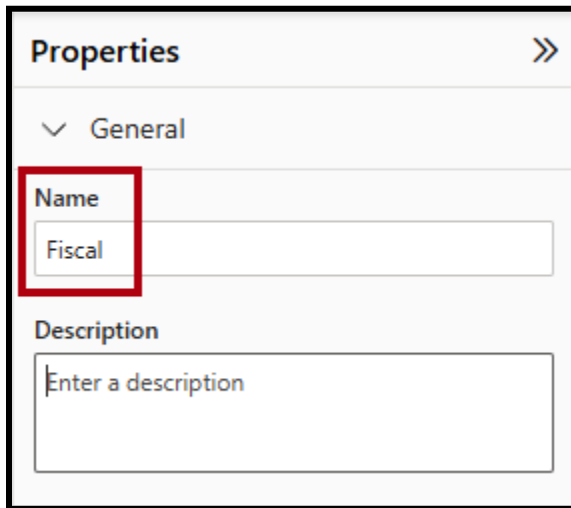
Complete the Date table

In this task, you'll complete the design of the **Date table** by hiding a column and creating a **hierarchy**. You'll then create **relationships** to the **Sales** and **Targets** tables.

1. Switch to **Model view**.
2. In the **Date table**, hide the **MonthKey** column (set Is Hidden to **Yes**).
3. In the **Data pane**, select the Date table, right-click the Year column, and then select **Create hierarchy**.

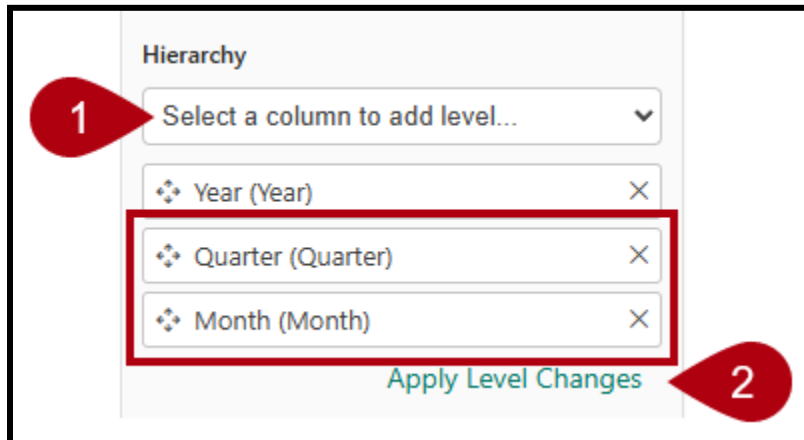


4. In the **Properties** pane, in the Name box, replace the value with **Fiscal**.



The screenshot shows the 'Properties' pane with the 'General' tab selected. The 'Name' field is highlighted with a red box and contains the text 'Fiscal'. Below it is a 'Description' field with the placeholder text 'Enter a description'.

5. Two add levels to the hierarchy, in the Hierarchy dropdown list, select Quarter and then select Month, and then select Apply Level Changes.



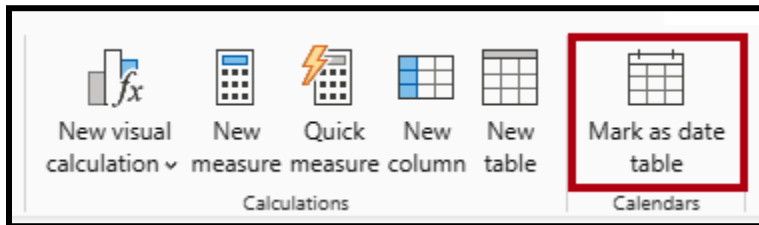
The screenshot shows the 'Hierarchy' pane. A red circle with the number '1' points to the dropdown menu 'Select a column to add level...'. Below the dropdown, a list of levels is shown: 'Year (Year)', 'Quarter (Quarter)', and 'Month (Month)'. The 'Quarter (Quarter)' and 'Month (Month)' levels are highlighted with a red box. A red circle with the number '2' points to the 'Apply Level Changes' button at the bottom right.

6. Create the following **two model relationships**:
- Date | Date to Sales | OrderDate
 - Date | Date to Targets | TargetMonth
7. Hide the following two columns:
- Sales | OrderDate
 - Targets | TargetMonth

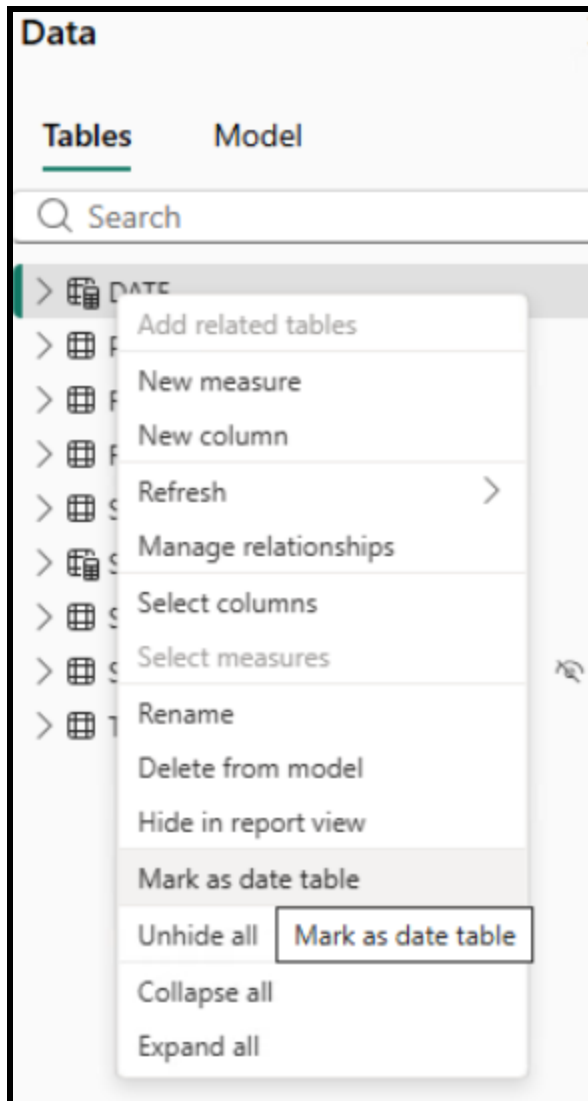
Mark the Date table

In this task, you'll mark the Date table as a date table.

1. Switch to **Report view**.
2. In the **Data pane**, select the **Date table** (not the Date field).
3. On the Table Tools contextual ribbon, from inside the Calendars group, select Mark as **Date Table**.

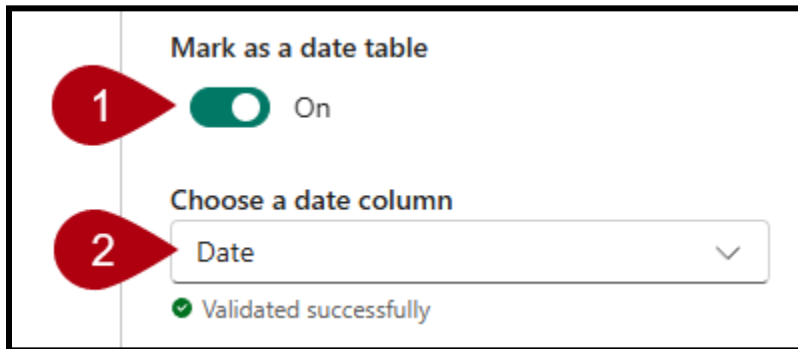


Or if you find it there, right click Date Table and select Mark as a Date Table



4. In the **Mark as a Date Table** window, slide the Mark as a Date Table property to **On**.

5. In the Choose a date column dropdown list, select Date.



6. Select **Save**.

7. Save the Power BI Desktop file.

Power BI Desktop now understands that this table defines date (time).

This design approach for a date table is suitable when you don't have a date table in your data source. If you have a data warehouse, it would be appropriate to load date data from its date dimension table rather than "redefining" date logic in your data model.

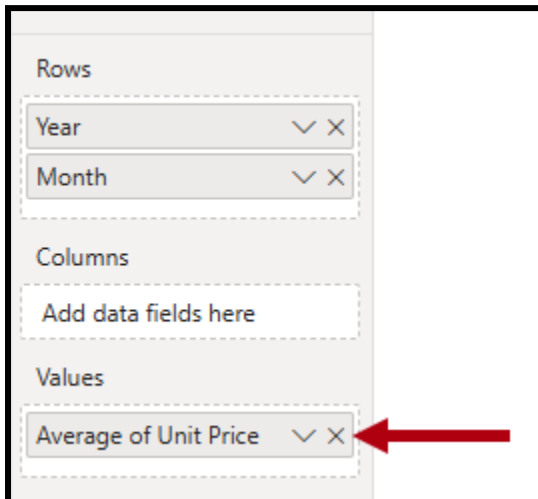
Create simple measures

In this task, you'll create **simple measures**. Simple measures aggregate values in a single column or count rows of a table.

1. In **Report view**, on Page 2, from the Data pane, drag the **Sales | Unit Price** field into the matrix visual.

Year	Average of Unit Price
FY2018	\$748.68
2017 Aug	\$758.93
2017 Dec	\$734.58
2017 Jul	\$655.59
2017 Nov	\$752.31
2017 Oct	\$677.45

2. In the visual fields pane (located in the Visualizations pane), in the Values well, notice that Unit Price field is configured as Average of Unit Price.



3. Select the down-arrow for Average of Unit Price, and then notice the available menu options.

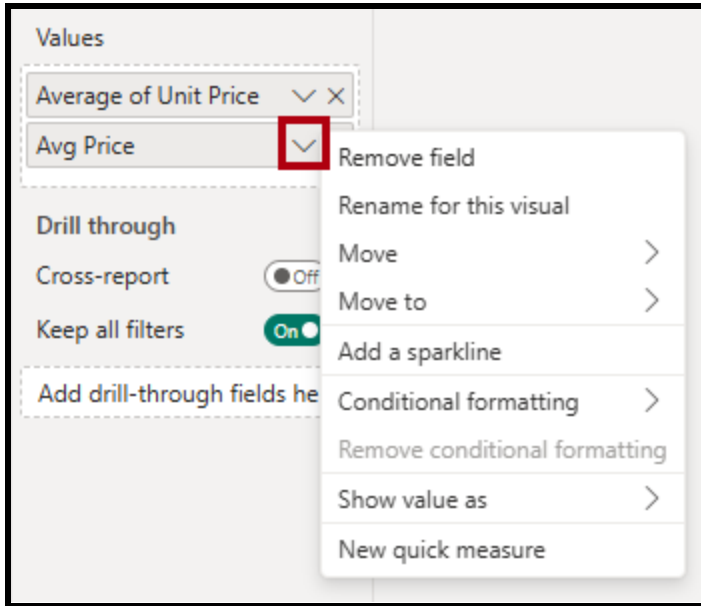
Visible numeric columns allow report authors at report design time to decide how column values will summarize (or not). However, it can result in inappropriate reporting.

Some data modelers don't like leaving things to chance, so they choose to hide these columns and instead expose aggregation logic defined in measures. It's the approach you'll now take in this lab.

4. To create a measure, in the Data pane, right-click the Sales table, and then select New Measure.
5. In the formula bar, add the following measure definition:

Avg Price = AVERAGE(Sales[Unit Price])

6. Add the Avg Price measure to the matrix visual, and notice that it produces the same result as the Unit Price column (but with different formatting).
7. In the Values well, open the context menu for the Avg Price field, and notice that it isn't possible to change the aggregation technique.



It's not possible to modify the aggregation behavior of a measure.

8. Use the snippets file definitions to create the following five measures for the Sales table:
 - Median Price
 - Min Price
 - Max Price
 - Orders
 - Order Lines

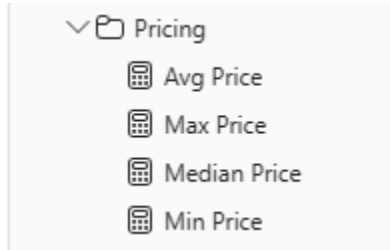
*The **DISTINCTCOUNT** function used in the Orders measure counts orders only once (ignoring duplicates).*

*The **COUNTROWS** function used in the Order Lines measure operates over a table.*

In this case, the number of orders is calculated by counting the distinct SalesOrderNumber column values, while the number of order lines is simply the number of table rows (each row is a line of an order).

9. Switch to Model view, and then multi-select the four price measures: Avg Price, Max Price, Median Price, and Min Price.
10. For the multi-selection of measures, configure the following requirements:

- Set the format to two decimal places.
- Assign to a display folder named *Pricing* (use the Display folder property in the Properties pane).

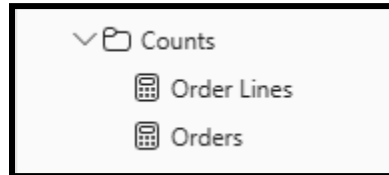


11. Hide the Unit Price column.

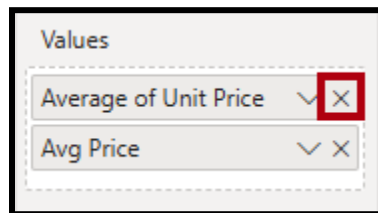
The Unit Price column is no longer available to report authors. They must use the pricing measures you've added to the model. This design approach ensures that report authors won't inappropriately aggregate prices, for example, by summing them.

12. Multi-select the Order Lines and Orders measures, and then configure the following requirements:

- Set the format use the thousands separator.
- Assign to a display folder named *Counts*.



13. In Report view, in the Values well of the matrix visual, for Average of Unit Price, select X to remove it.



14. Increase the size of the matrix visual to fill the page width and height.

15. Add the following five measures to the matrix visual:

- Median Price
- Min Price
- Max Price

- Orders
 - Order Lines
16. Verify that the results look sensible and are correctly formatted.

Year	Avg Price	Median Price	Min Price	Max Price	Orders	Order Lines
 FY2018	\$748.68	\$419.46	\$4.75	\$2,146.96	739	8,459
2017 Jul	\$655.59	\$419.46	\$5.19	\$2,146.96	38	352
2017 Aug	\$758.93	\$419.46	\$4.75	\$2,146.96	75	785
2017 Sep	\$741.85	\$419.46	\$5.19	\$2,146.96	60	593
2017 Oct	\$677.45	\$419.46	\$5.19	\$2,146.96	40	499
2017 Nov	\$752.31	\$419.46	\$5.01	\$2,146.96	90	1,106
2017 Dec	\$734.58	\$419.46	\$5.01	\$2,146.96	63	803

Create additional measures

In this task, you'll create more measures that use more complex formulas.

1. In Report view, select Page 1 and review the table visual of salespeople (on the right), noticing the total for the Sum of Target

column.

Salesperson	Sum of Sales	Sum of Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsoflias	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Rachel Valdez	\$1,877,743	\$4,125,000
Ranjit Varkey Chudukatil	\$4,527,840	\$9,050,000
Shu Ito	\$18,001,116	\$59,850,000
Stephen Jiang	\$65,868,919	\$110,150,000
Syed Abbas	\$1,391,025	\$3,050,000
Tete Mensa-Annan	\$12,004,822	\$17,100,000
Tsvi Reiter	\$7,638,607	\$13,250,000
Total	\$77,548,570	\$676,210,000

2. Select the table visual, and then in the Visualizations pane, remove Sum of Target.
3. Rename the Targets | Target column as *TargetAmount*.
Tip: There are several ways to rename the column in Report view: In the Data pane, you can right-click the column, and then select Rename—or, double-click the column, or press F2.
4. Create the following measure on the Targets table:

Target =

IF(

HASONEVALUE('Salesperson (Performance)'[Salesperson]),

SUM(Targets[TargetAmount])

)

5. The *HASONEVALUE* function tests whether a single value in the Salesperson column is filtered. When true, the expression returns the

sum of target amounts (for just that salesperson). When false, BLANK is returned.

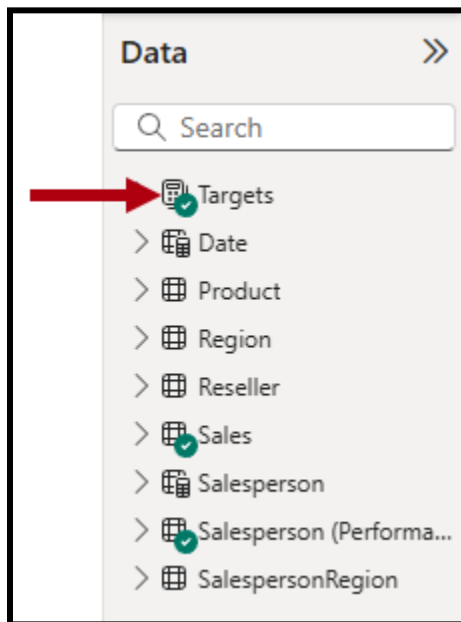
6. Format the Target measure for zero decimal places.

Tip: You can use the Measure Tools contextual ribbon.

7. Hide the TargetAmount column.

Tip: You can right-click the column in the Data pane, and then select Hide.

8. Notice that the Targets table now appears at the top of the list.



Tables that comprise only visible measures are automatically listed at the top of the list.

9. Add the Target measure to the table visual.

10. Notice that the Target column total is now BLANK.

Salesperson	Sum of Sales	Target
Amy Alberts	\$10,288,626	\$19,450,000
Brian Welcker	\$77,548,570	\$221,700,000
David Campbell	\$12,004,822	\$19,625,000
Garrett Vargas	\$13,875,633	\$23,675,000
Jae Pak	\$8,410,883	\$13,575,000
Jillian Carson	\$7,633,387	\$13,675,000
José Saraiva	\$13,875,633	\$18,875,000
Linda Mitchell	\$25,634,503	\$40,850,000
Lynn Tsoflias	\$1,391,025	\$3,210,000
Michael Blythe	\$21,987,348	\$31,150,000
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000
Rachel Valdez	\$1,877,743	\$4,125,000
Ranjit Varkey Chudukatil	\$4,527,840	\$9,050,000
Shu Ito	\$18,001,116	\$59,850,000
Stephen Jiang	\$65,868,919	\$110,150,000
Syed Abbas	\$1,391,025	\$3,050,000
Tete Mensa-Annan	\$12,004,822	\$17,100,000
Tsvi Reiter	\$7,638,607	\$13,250,000
Total	\$77,548,570	

11. Use the snippets file definitions to create the following two measures for the Targets table:
- Variance
 - Variance Margin
12. Format the Variance measure for zero decimal places.
13. Format the Variance Margin measure as percentage with two decimal places.
14. Add the Variance and Variance Margin measures to the table visual.

15. Resize the table visual so all columns and rows can be seen.

Salesperson	Sum of Sales	Target	Variance	Variance Margin
Amy Alberts	\$10,288,626	\$19,450,000	-\$9,161,374	-47.10%
Brian Welcker	\$77,548,570	\$221,700,000	-\$144,151,430	-65.02%
David Campbell	\$12,004,822	\$19,625,000	-\$7,620,178	-38.83%
Garrett Vargas	\$13,875,633	\$23,675,000	-\$9,799,367	-41.39%
Jae Pak	\$8,410,883	\$13,575,000	-\$5,164,117	-38.04%
Jillian Carson	\$7,633,387	\$13,675,000	-\$6,041,613	-44.18%
José Saraiva	\$13,875,633	\$18,875,000	-\$4,999,367	-26.49%
Linda Mitchell	\$25,634,503	\$40,850,000	-\$15,215,497	-37.25%
Lynn Tsoflias	\$1,391,025	\$3,210,000	-\$1,818,975	-56.67%
Michael Blythe	\$21,987,348	\$31,150,000	-\$9,162,652	-29.41%
Pamela Ansman-Wolfe	\$30,005,939	\$53,850,000	-\$23,844,061	-44.28%
Rachel Valdez	\$1,877,743	\$4,125,000	-\$2,247,257	-54.48%
Ranjit Varkey Chudukatil	\$4,527,840	\$9,050,000	-\$4,522,160	-49.97%
Shu Ito	\$18,001,116	\$59,850,000	-\$41,848,884	-69.92%
Stephen Jiang	\$65,868,919	\$110,150,000	-\$44,281,081	-40.20%
Syed Abbas	\$1,391,025	\$3,050,000	-\$1,658,975	-54.39%
Tete Mensa-Annan	\$12,004,822	\$17,100,000	-\$5,095,178	-29.80%
Tsvi Reiter	\$7,638,607	\$13,250,000	-\$5,611,393	-42.35%
Total	\$77,548,570			

While it appears all salespeople aren't meeting target, remember that the table visual isn't yet filtered by a specific time period. You'll produce sales performance reports that filter by a user-selected time period in the Design a Report in Power BI Desktop lab.

16. Save the Power BI Desktop file.

Lab complete

You may choose to save your Power BI report, though it's not necessary for this lab. In the next exercise, you'll work with a pre-made starter file.

1. Navigate to the "File" menu in the top left corner and select "Save As".
2. Select Browse this device.
3. Select the folder where you want to save the file and give it a descriptive name.

4. Select the Save button to save your report as a .pbix file.
5. If a dialog box appears prompting you to apply pending query changes, select Apply.
6. Close Power BI Desktop.